

Orientação a Objetos – Classes Abstratas, Interfaces e Tratamento de Exceções



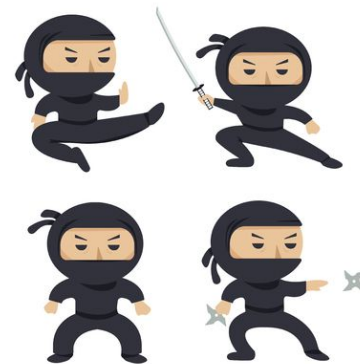
Classes Abstratas

- Uma classe abstrata é uma classe que serve de modelo/base para outras classes. Ela não pode ser instanciada diretamente, ou seja, você não pode criar objetos a partir dela;
- Ela define métodos obrigatórios que devem ser implementados pelas subclasses. Isso é muito útil quando você quer garantir que todas as subclasses tenham determinados métodos com um certo nome e assinatura.



Classes Abstratas

- Imagine que você está criando um jogo com vários tipos de personagens. Todos os personagens devem poder se mover e atacar, mas cada um faz isso de forma diferente;
- Você pode criar uma classe abstrata Personagem com os métodos mover() e atacar() definidos como abstratos. Assim, qualquer classe que herdar de Personagem será obrigada a implementar esses métodos.



Classes Abstratas

- Em Python, usamos o módulo abc (Abstract Base Classes) para criar classes abstratas: `from abc import ABC, abstractmethod`.
- ABC é a classe base para classes abstratas;
- `@abstractmethod` é o decorador que marca um método como abstrato.

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC): # Herda de ABC, tornando-se  
    abstrata
```

```
    @abstractmethod  
    def emitir_som(self):  
        pass
```

```
    @abstractmethod  
    def mover(self):  
        pass
```

Classes Abstratas

```
class Cachorro(Animal):
```

```
    def emitir_som(self):  
        return "Au au!"
```

```
    def mover(self):  
        return "Correndo com as patas"
```

```
class Peixe(Animal):
```

```
    def emitir_som(self):  
        return "Blublublu"
```

```
    def mover(self):  
        return "Nadando"
```

```
c = Cachorro()  
print(c.emitir_som()) # Saída: Au au!
```


Classes Abstratas

- Pense em uma classe abstrata como um contrato. Ela diz: "Qualquer classe que me herdar deve implementar esses métodos. Se não fizer isso, não pode ser usada."

```
from abc import ABC, abstractmethod
```

```
class Pagamento(ABC):
```

```
    @abstractmethod
    def pagar(self, valor):
        pass
```

```
class CartaoCredito(Pagamento):
```

```
    def pagar(self, valor):
        print(f"Pagando R${valor:.2f} com cartão de crédito.")
```

```
class Boleto(Pagamento):
```

```
    def pagar(self, valor):
        print(f"Gerando boleto no valor de
R${valor:.2f}.")
```

```
p = CartaoCredito()
p.pagar(100.0) # Saída: Pagando R$100.00
com cartão de crédito.
```

Classes Abstratas

- Se você esquecer de implementar algum método abstrato em uma subclasse, o Python não permitirá que você instancie essa subclasse (a subclasse será uma classe abstrata).

```
class Gato(Animal):  
    def emitir_som(self):  
        return "Miau" # esqueci de implementar `mover()`
```

```
g = Gato() # ERRO: TypeError: Can't instantiate abstract class Gato
```

Classes Abstratas

- Se pelo menos um método for marcado com `@abstractmethod`, a classe se torna abstrata automaticamente, mesmo que ela herde de uma classe abstrata.

```
class Exemplo(ABC):  
    @abstractmethod  
    def teste(self):  
        pass
```


Classes Abstratas

- Uma classe abstrata pode ter:
 - Métodos abstratos (com `@abstractmethod`);
 - Métodos concretos (implementados normalmente).

```
class Pessoa(ABC):
```

```
    @abstractmethod  
    def falar(self):  
        pass
```

```
    def respirar(self):  
        print("Pessoa respirando") # método concreto
```

Classes Abstratas

- Em Python, o construtor `__init__` pode ser abstrato. Isso força subclasses a implementarem seus próprios construtores. Isso é válido, embora raro e não recomendado em todos os casos.

```
class Produto(ABC):  
    @abstractmethod  
    def __init__(self, nome):  
        pass
```

Classes Abstratas

- Em Python, é permitido declarar métodos estáticos como abstratos, desde que se use os dois decoradores corretamente: `@abstractmethod` acima de `@staticmethod`. A ordem dos decoradores importa!

```
class Exemplo(ABC):  
    @staticmethod  
    @abstractmethod  
    def metodo_estatico():  
        pass
```

Classes Abstratas

- Você pode criar métodos abstratos privados (com nomes iniciados com dois underscores), mas o uso se torna mais complicado por causa do name mangling. Isso funciona, mas não é recomendado, pois complica a herança e legibilidade. Portanto, tecnicamente é possível, mas não é boa prática.

```
class Teste(ABC):  
    @abstractmethod  
    def __escondido(self):  
        pass
```

Classes Abstratas

Vantagem	Descrição
✓ Padronização	Garante que todas as subclasses implementem os métodos obrigatórios.
✓ Reutilização de código	Métodos concretos na classe abstrata podem ser reutilizados pelas subclasses.
✓ Facilidade de manutenção	Ao mudar a interface abstrata, o sistema aponta as subclasses que precisam se adaptar.
✓ Melhoria do design (SOLID)	Apoia princípios como substituição de Liskov e segregação de interface.
✓ Suporte a polimorfismo	Permite tratar objetos diferentes de forma genérica, por meio da classe abstrata.
✓ Facilidade nos testes (mocking)	Define claramente os métodos esperados, facilitando a criação de testes com mocks.
✓ Organização de sistemas grandes	Ajuda a estruturar e agrupar classes com comportamentos semelhantes.

Classes Abstratas

```
from abc import ABC, abstractmethod

class Cavaleiro(ABC):
    def __init__(self, nome, signo, golpe_especial):
        self.nome = nome
        self.signo = signo
        self.golpe_especial = golpe_especial

    def apresentar(self):
        print(f"Eu sou {self.nome}, Cavaleiro de {self.signo}!")

    @abstractmethod
    def usar_golpe_especial(self):
        pass
```


Classes Abstratas

```
class CavaleiroDeBronze(Cavaleiro):
    def usar_golpe_especial(self):
        print(f"{self.nome} usa o golpe: {self.golpe_especial}!")

class CavaleiroDeOuro(Cavaleiro):
    def usar_golpe_especial(self):
        print(f"{self.nome} invoca o golpe supremo: {self.golpe_especial}!")

seiya = CavaleiroDeBronze("Seiya", "Pégaso", "Meteoro de Pégaso")
shaka = CavaleiroDeOuro("Shaka", "Virgem", "Tesouro do Céu")

seiya.apresentar()
seiya.usar_golpe_especial()

shaka.apresentar()
shaka.usar_golpe_especial()
```

Exercício

- Você foi contratado para desenvolver um sistema para gerenciar informações e estatísticas dos clubes participantes da Copa do Mundo de Clubes da FIFA. Para isso, implemente uma classe abstrata chamada `ClubeParticipante` com os atributos `nome`, `pais`, `confederacao`, `ranking_fifa`, `gols_marcados` e `vitorias`, incluindo um método concreto `exibir_dados()` para mostrar os dados do clube, e métodos abstratos `calcular_desempenho()` — que deve calcular um índice baseado na fórmula específica da confederação (para clubes da UEFA: $\text{desempenho} = \text{vitórias} \times 3 + \text{gols marcados} \times 0,5$; para clubes da CONMEBOL: $\text{desempenho} = \text{vitórias} \times 3 + \text{gols marcados} \times 0,7$) — e `gerar_relatorio_tecnico()`, que deve exibir as informações básicas do clube junto com o desempenho calculado. Crie duas subclasses concretas, `ClubeUEFA` e `ClubeCONMEBOL`, que implementem esses métodos de acordo com as regras definidas, e teste a aplicação criando instâncias de cada tipo, demonstrando o uso de abstração, herança e polimorfismo em Python.



Explicando o código - Sistema de clubes de futebol

Importação ABC (Abstract Base Class)

```
1 from abc import ABC, abstractmethod
```

ABC: Classe base para criar classes abstratas

abstractmethod: Decorador para métodos que devem ser implementados pelas classes filhas

```
class ClubeParticipante(ABC):
```

Características:

- **Herda de ABC**: Torna-se uma classe abstrata (não pode ser instanciada diretamente)
- **Construtor (`__init__`)**: Define atributos comuns a todos os clubes
 - `nome`, `pais`, `confederacao`, `ranking_fifa`, `gols_marcados`, `vitorias`

Explicando o código - Sistema de clubes de futebol

Métodos

Método concreto

```
12     def exibir_dados(self):
```

Implementado na classe base

Exibe informações básicas do clube

Pode ser usado por todas as classes filhas

Métodos Abstratos

```
20     @abstractmethod
21     def calcular_desempenho(self):
22         pass
23
24     @abstractmethod
25     def gerar_relatorio_tecnico(self):
26         pass
```

Devem ser implementados pelas classes filhas

Força cada confederação a ter sua própria lógica de cálculo

Explicando o código - Sistema de clubes de futebol

Método Concreto X Método Abstrato

Método Concreto

```
12     def exibir_dados(self):
13         print(f"Nome: {self.nome}")
14         print(f"País: {self.pais}")
15         print(f"Confederação: {self.confederacao}")
16         print(f"Ranking FIFA: {self.ranking_fifa}")
17         print(f"Gols Marcados: {self.gols_marcados}")
18         print(f"Vitórias: {self.vitorias}")
19
```

Este é um **método concreto**. Vamos entender o porquê:

- **Ele tem uma implementação completa:** Dentro do `def`, existe um corpo com código que executa uma tarefa específica. As linhas `print(...)` são as instruções. Ele sabe exatamente o que fazer: imprimir os dados de um objeto na tela.
- **Ele é "pronto para usar":** Qualquer classe que possua este método pode chamá-lo diretamente, e ele funcionará da mesma maneira para todas, pois a sua lógica já está definida e finalizada.

Analogia: Pense em um manual de instruções para montar um móvel. O método concreto é o passo a passo detalhado, com diagramas e texto, que diz exatamente onde cada parafuso vai. Você simplesmente segue as instruções.

No exemplo acima, a ação de "exibir dados" é genérica e provavelmente será a mesma para qualquer tipo de seleção ou time. Portanto, faz sentido que ela seja concreta, oferecendo uma funcionalidade comum e reutilizável.

Explicando o código - Sistema de clubes de futebol

Método Concreto X Método Abstrato

Métodos Abstratos

```
20     @abstractmethod
21     def calcular_desempenho(self):
22         pass
23
24     @abstractmethod
25     def gerar_relatorio_tecnico(self):
26         pass
```

Estes são **métodos abstratos**. As características que os definem são:

- **Eles não têm implementação:** O corpo do método é `pass`, uma palavra-chave em Python que significa "não faça nada". O método existe, tem um nome e parâmetros, mas não há código que execute uma tarefa.
- **Eles são um contrato (uma obrigação):** A anotação `@abstractmethod` acima deles é a chave. Ela transforma o método em uma regra. A classe que contém esses métodos (que deve ser uma classe abstrata, como por exemplo `Selecao`) está dizendo: "Eu não sei *como* calcular o desempenho ou gerar um relatório técnico de uma forma genérica, pois isso pode variar muito. Mas eu **obrigo** que qualquer classe que me herde (uma classe filha, como `SelecaoBrasileira` ou `SelecaoAlema`) crie a sua própria lógica para estes métodos."

Analogia: Voltando ao manual, o método abstrato é um capítulo obrigatório com o título "Inspeção de Segurança Final", mas cujo conteúdo está em branco. O manual está te forçando a realizar essa inspeção, mas você (a classe filha) precisa definir e escrever os passos de como você vai fazê-la, pois pode depender do tipo de móvel que está montando.

Método Concreto X Método Abstrato

Tabela comparativa

Característica	Método Concreto	Método Abstrato
Implementação	Possui um corpo com código.	Não possui um corpo de código.
Localização	Pode existir em qualquer classe.	Só pode existir em classes abstratas.
Obrigatoriedade	Não obriga a implementação nas subclasses.	Obriga as subclasses concretas a implementá-lo.
Finalidade	Executar uma tarefa específica.	Definir um contrato de comportamento para as subclasses.

Classes Filhas (implementações Específicas)

Classe ClubeUEFA (Europa)

```
28 class ClubeUEFA(ClubeParticipante):  
29     def __init__(self, nome, pais, ranking_fifa, gols_marcados, vitorias):  
30         super().__init__(nome, pais, "UEFA", ranking_fifa, gols_marcados, vitori  
31
```

Construtor:

- Usa `super().__init__()` para chamar o construtor da classe pai
- Automaticamente define `confederacao="UEFA"`

Implementações específicas:

```
32     def calcular_desempenho(self):  
33         return self.vitorias * 3 + self.gols_marcados * 0.5  
34
```

Classes Filhas (implementações Específicas)

Classe ClubeUEFA (Europa)

```
28 class ClubeUEFA(ClubeParticipante):  
29     def __init__(self, nome, pais, ranking_fifa, gols_marcados, vitorias):  
30         super().__init__(nome, pais, "UEFA", ranking_fifa, gols_marcados, vitori  
31
```

Construtor:

- Usa `super().__init__()` para chamar o construtor da classe pai
- Automaticamente define `confederacao="UEFA"`

Implementações específicas:

```
32     def calcular_desempenho(self):  
33         return self.vitorias * 3 + self.gols_marcados * 0.5  
34
```

- Fórmula CONMEBOL: Gols valem mais ($\times 0.7$) que na UEFA
- Reflete estilo sul-americano mais ofensivo

Classes Filhas (implementações Específicas)

Classe ClubeUEFA (Europa)

```
28 class ClubeUEFA(ClubeParticipante):  
29     def __init__(self, nome, pais, ranking_fifa, gols_marcados, vitorias):  
30         super().__init__(nome, pais, "UEFA", ranking_fifa, gols_marcados, vitori  
31
```

Construtor:

- Usa `super().__init__()` para chamar o construtor da classe pai
- Automaticamente define `confederacao="UEFA"`

Implementações específicas:

```
32     def calcular_desempenho(self):  
33         return self.vitorias * 3 + self.gols_marcados * 0.5  
34
```

- Fórmula CONMEBOL: Gols valem mais ($\times 0.7$) que na UEFA
- Reflete estilo sul-americano mais ofensivo

Explicando o código - Sistema de clubes de futebol

Conceitos de POO demonstrados

1. Herança

- Classes filhas herdam atributos e métodos da classe pai
- `super().__init__()` chama o construtor da classe pai

2. Abstração

- Classe abstrata define "contrato" que as filhas devem seguir
- Não pode ser instanciada diretamente

3. Polimorfismo

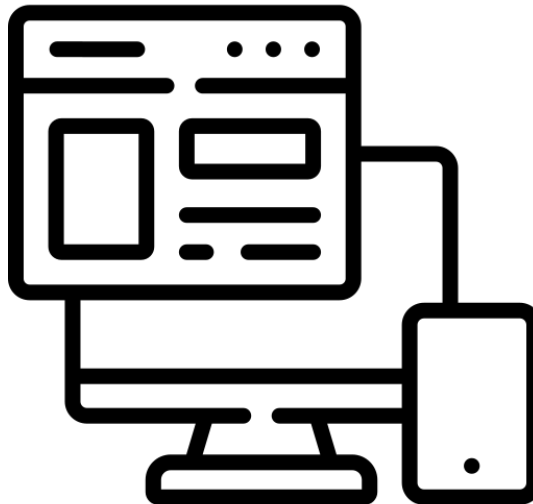
- Cada classe implementa os métodos abstratos de forma diferente
- Mesmo método (`calcular_desempenho`) com comportamentos distintos

4. Encapsulamento

- Atributos organizados dentro das classes
- Métodos controlam como os dados são acessados

Interfaces

- Python não tem uma palavra-chave específica como interface (como em Java ou C#). Em vez disso, utiliza classes abstratas (do módulo abc) como forma oficial de definir interfaces;
- Em Python, uma interface é representada por uma classe abstrata contendo apenas métodos abstratos.



Interfaces

- Essa classe funciona como uma interface porque:
 - Não tem implementação concreta;
 - Obriga qualquer subclasse a implementar todos os métodos.

```
from abc import ABC, abstractmethod
```

```
class MinhaInterface(ABC):
```

```
    @abstractmethod
```

```
    def metodo1(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def metodo2(self, valor):
```

```
        pass
```


Interfaces

```
from abc import ABC, abstractmethod
```

```
class PessoaEscola(ABC):
```

```
    @abstractmethod  
    def identificar(self):  
        pass
```

```
    @abstractmethod  
    def realizar_atividade(self):  
        pass
```

```
class Professor(PessoaEscola):  
    def __init__(self, nome, disciplina):  
        self.nome = nome  
        self.disciplina = disciplina
```

```
    def identificar(self):  
        print(f"Professor: {self.nome}, Disciplina:  
{self.disciplina}")
```

```
    def realizar_atividade(self):  
        print(f"{self.nome} está dando aula de  
{self.disciplina}.")
```

Interfaces

```
class Aluno(PessoaEscola):
    def __init__(self, nome, serie):
        self.nome = nome
        self.serie = serie

    def identificar(self):
        print(f"Aluno: {self.nome}, Série: {self.serie}")

    def realizar_atividade(self):
        print(f"{self.nome} está estudando para a prova.")

class Funcionario(PessoaEscola):
    def __init__(self, nome, setor):
        self.nome = nome
        self.setor = setor

    def identificar(self):
        print(f"Funcionário: {self.nome}, Setor: {self.setor}")

    def realizar_atividade(self):
        print(f"{self.nome} está trabalhando no setor de {self.setor}.")
```

Interfaces

```
def rotina(pessoa: PessoaEscola):  
    pessoa.identificar()  
    pessoa.realizar_atividade()  
    print("-" * 40)
```

```
# Lista de pessoas  
pessoas = [  
    Professor("Ana", "Matemática"),  
    Aluno("Carlos", "8º ano"),  
    Funcionario("João", "Secretaria")  
]
```

```
# Testando  
for p in pessoas:  
    rotina(p)
```

Tratamento de Exceções

- Uma exceção é um erro que ocorre durante a execução do programa. Quando algo dá errado (ex: dividir por zero, acessar índice inválido, ler um arquivo inexistente), o Python lança uma exceção e, se ela não for tratada, o programa interrompe a execução com uma mensagem de erro.



Tratamento de Exceções

- Usamos a estrutura try / except:

try:

código que pode causar erro

except TipoDeErro:

o que fazer se esse erro acontecer

try:

x = int(input("Digite um número: "))

resultado = 10 / x

print(f"Resultado: {resultado}")

except ZeroDivisionError:

print("Não é possível dividir por zero!")

- O código dentro de try é executado. Se ocorrer um erro do tipo ZeroDivisionError, o código dentro do except é executado.

Tratamento de Exceções

- Você pode adicionar:
 - else: executado se não ocorrer exceção;
 - finally: executado sempre, com ou sem exceção.

```
try:
    x = int(input("Digite um número: "))
    resultado = 10 / x
except ZeroDivisionError:
    print("Erro: Divisão por zero!")
else:
    print(f"Resultado: {resultado}")
finally:
    print("Operação finalizada.")
```


Tratamento de Exceções

- Múltiplos tipos de erro:

```
try:
    a = int(input("Digite um número: "))
    b = int(input("Digite outro número: "))
    print(f"Resultado: {a / b}")
except ValueError:
    print("Erro: Você deve digitar apenas números.")
except ZeroDivisionError:
    print("Erro: Não pode dividir por zero.")
```

Tratamento de Exceções

Exceção (Erro)	Quando acontece	Exemplo de código	Mensagem de erro (típica)
<code>ZeroDivisionError</code>	Quando tenta dividir por zero	<code>10 / 0</code>	<code>division by zero</code>
<code>ValueError</code>	Conversão inválida de tipo, com valor inadequado	<code>int("abc")</code>	<code>invalid literal for int()</code>
<code>TypeError</code>	Operações com tipos incompatíveis	<code>"abc" + 5</code>	<code>can only concatenate str and int</code>
<code>IndexError</code>	Índice fora do limite da lista ou tupla	<code>lista = [1]; lista[5]</code>	<code>list index out of range</code>
<code>KeyError</code>	Acesso a uma chave inexistente em um dicionário	<code>d = {"a": 1}; d["b"]</code>	<code>'b'</code>
<code>NameError</code>	Variável não definida usada no código	<code>print(x)</code> (sem declarar antes)	<code>name 'x' is not defined</code>
<code>FileNotFoundError</code>	Arquivo ou caminho inexistente	<code>open("nada.txt")</code>	<code>No such file or directory</code>

Tratamento de Exceções

AttributeError	Uso de método ou atributo que não existe	<code>"abc".voar()</code>	<code>'str' object has no attribute 'voar'</code>
ImportError	Módulo ou função não encontrada na importação	<code>from math import raiz</code>	<code>cannot import name 'raiz'</code>
IndentationError	Erro de indentação (recuo) no código	<code>def f():\nprint("oi")</code>	<code>expected an indented block</code>
SyntaxError	Código com erro de sintaxe	<code>if True print("oi")</code>	<code>invalid syntax</code>
RuntimeError	Erro em tempo de execução que não se encaixa em outro tipo	Exceções genéricas em execução	<code>RuntimeError</code>
StopIteration	Fim de um iterador (ex: <code>next()</code> em iterador vazio)	<code>next(iter([]))</code>	<code>StopIteration</code>
Exception	Classe base para capturar qualquer exceção tratável	<code>except Exception as e:</code>	Varia de acordo com o erro real

Tratamento de Exceções

- Se você não sabe exatamente qual erro pode ocorrer, pode usar um tratamento genérico;
- É quando você captura qualquer tipo de erro, sem especificar um tipo como `ZeroDivisionError` ou `ValueError`;
- `Exception` é a classe base de quase todos os erros normais.

```
try:  
    x = int("abc")  
except Exception as erro:  
    print("Erro genérico capturado:", erro)
```

```
try:  
    arquivo = open("inexistente.txt")  
except Exception as erro:  
    print(f'Ocorreu um erro: {erro}')
```

Tratamento de Exceções

- Quando usar o tratamento genérico?
 - Ao capturar vários tipos possíveis de erro, e você quer uma mensagem padrão;
 - Em programas robustos com log de erros;
 - Em sistemas onde interromper o programa não é uma opção (ex: servidores, sistemas críticos).



Tratamento de Exceções

- O comando raise serve para lançar uma exceção de forma proposital. Você usa raise quando quer interromper a execução do programa se uma condição indesejada acontecer;
- Sintaxe: raise TipoDeErro("mensagem opcional").

```
def registrar_nota(nome_aluno, nota):  
    if nota < 0 or nota > 10:  
        raise ValueError(f"A nota de {nome_aluno} está fora do intervalo permitido (0 a 10).")  
    print(f"Nota {nota} registrada com sucesso para o aluno {nome_aluno}.")
```

```
try:  
    nome = input("Digite o nome do aluno: ")  
    nota = float(input("Digite a nota do aluno (0 a 10): "))
```

```
    registrar_nota(nome, nota)
```

```
except ValueError as erro:  
    print("Erro:", erro)
```


Tratamento de Exceções

```
class ContaBancaria:
```

```
    def __init__(self, titular, saldo_inicial=0.0):
```

```
        self.titular = titular
```

```
        self.saldo = saldo_inicial
```

```
    def ver_saldo(self):
```

```
        print(f"\nSaldo atual de {self.titular}: R${self.saldo:.2f}")
```

```
    def depositar(self, valor):
```

```
        if valor <= 0:
```

```
            raise ValueError("O valor do depósito deve ser positivo.")
```

```
        self.saldo += valor
```

```
        print(f"Depósito de R${valor:.2f} realizado com sucesso.")
```

```
    def sacar(self, valor):
```

```
        if valor <= 0:
```

```
            raise ValueError("O valor do saque deve ser positivo.")
```

```
        if valor > self.saldo:
```

```
            raise Exception("Saldo insuficiente para saque.")
```

```
        self.saldo -= valor
```

```
        print(f"Saque de R${valor:.2f} realizado com sucesso.")
```

Tratamento de Exceções

```
class SistemaBancario:
```

```
    def __init__(self):
```

```
        nome = input("Digite o nome do titular da conta: ")
```

```
        self.conta = ContaBancaria(nome, saldo_inicial=1000.0)
```

```
    def exibir_menu(self):
```

```
        while True:
```

```
            try:
```

```
                print("\n--- Menu Bancário ---")
```

```
                print("1. Ver saldo")
```

```
                print("2. Depositar")
```

```
                print("3. Sacar")
```

```
                print("4. Sair")
```

```
            opcao = int(input("Escolha uma opção: "))
```

```
            if opcao == 1:
```

```
                self.conta.ver_saldo()
```

```
            elif opcao == 2:
```

```
                valor = float(input("Digite o valor para depósito: "))
```

```
                self.conta.depositar(valor)
```

Tratamento de Exceções

```
elif opcao == 3:  
    valor = float(input("Digite o valor para saque: "))  
    self.conta.sacar(valor)
```

```
elif opcao == 4:  
    print("Saindo do sistema. Até logo!")  
    break
```

```
else:  
    print("Opção inválida. Escolha entre 1 e 4.")
```

```
except Exception as erro:  
    print(f"\nOcorreu um erro: {erro}")
```

Execução do sistema

```
sistema = SistemaBancario()  
sistema.exibir_menu()
```

Tratamento de Exceções

- Você pode criar suas próprias exceções para lidar com regras específicas (cria herdando da classe Exception);
- Por que criar exceções personalizadas? Para representar regras de negócio ou situações específicas do seu programa, com nomes mais claros e controle mais preciso.

```
class NotaInvalidaError(Exception):  
    pass  
  
def cadastrar_nota(nota):  
    if nota < 0 or nota > 10:  
        raise NotaInvalidaError("A nota deve estar entre 0 e 10.")  
    print("Nota registrada com sucesso!")  
  
try:  
    cadastrar_nota(12)  
except NotaInvalidaError as e:  
    print(f"Erro: {e}")
```

Tratamento de Exceções

Exceções personalizadas

```
class LimiteEmprestimosError(Exception):
```

```
    """Erro lançado quando o usuário ultrapassa o limite de empréstimos."""
```

```
    pass
```

```
class LivroIndisponivelError(Exception):
```

```
    """Erro lançado quando o livro já está emprestado."""
```

```
    pass
```

Classe Livro

```
class Livro:
```

```
    def __init__(self, titulo):
```

```
        self.titulo = titulo
```

```
        self.emprestado = False
```

Tratamento de Exceções

Classe Usuario

```
class Usuario:
```

```
    def __init__(self, nome):
```

```
        self.nome = nome
```

```
        self.livros_emprestados = []
```

```
    def emprestar_livro(self, livro):
```

```
        if len(self.livros_emprestados) >= 3:
```

```
            raise LimiteEmprestimosError(f"{self.nome} já possui 3 livros emprestados.")
```

```
        if livro.emprestado:
```

```
            raise LivroIndisponivelError(f"O livro '{livro.titulo}' já está emprestado.")
```

```
        livro.emprestado = True
```

```
        self.livros_emprestados.append(livro)
```

```
        print(f"{self.nome} emprestou o livro '{livro.titulo}' com sucesso.")
```


Tratamento de Exceções

Simulação

try:

livro1 = Livro("1984")

livro2 = Livro("Dom Casmurro")

livro3 = Livro("A Revolução dos Bichos")

livro4 = Livro("Capitães da Areia")

usuario = Usuario("Ana")

usuario.emprestar_livro(livro1)

usuario.emprestar_livro(livro2)

usuario.emprestar_livro(livro3)

usuario.emprestar_livro(livro4) # Deve lançar exceção

except LimiteEmprestimosError as e:

print("Erro de limite:", e)

except LivroIndisponivelError as e:

print("Erro de disponibilidade:", e)/

Tratamento de Exceções

Conceito

Finalidade

`except Exception as e:`

Captura **qualquer erro tratável** (forma segura do tratamento genérico).

Exceção personalizada

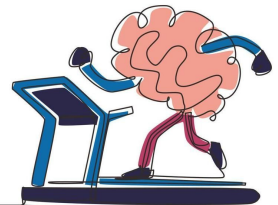
Permite criar **nomes de erro claros** para regras específicas.

`raise`

Serve para **lançar manualmente** uma exceção, comum com regras de negócio.

Exercício

- Implemente uma calculadora simples em Python que ofereça um menu para o usuário escolher entre as operações: (1) soma, (2) subtração, (3) multiplicação, (4) divisão e (5) sair. Para cada operação, o programa deve pedir dois números ao usuário e mostrar o resultado. Use blocos try/except para garantir que o programa não quebre caso o usuário digite valores inválidos (ex: letras em vez de números) ou tente dividir por zero. O programa deve continuar funcionando normalmente após qualquer erro, permitindo novas operações até o usuário escolher sair.



FIM

